Regal Research Team

INRIA Paris & LIP6, UPMC





INTERNSHIP REPORT

Persistent Data Structures using Non Volatile Memory

Student Name
Ajay Singh

Advisors:

Prof. Marc Shapiro Prof. Gael Thomas

Submission Date : 2017

Acknowledgement

Internship opportunity at Regal Lab at UPMC- LIP6&INRIA Paris was a great learning experience for me as I got to work on the very recent research topic of my field along with the best researchers and minds at work in this area. The experience at the lab personally and professionally was very rewarding. It has strengthened my belief that I can contribute to the research community and has convinced me to take up the challenging research problems and pursue a research degree as my future endeavour.

I am using this opportunity to express my deepest gratitude and special thanks to my internship advisor Dr Marc Shapiro, principal researcher(DR1) at Regal Lab, LIP6&INRIA for necessary advises, guidance and arranging all facilities to make life easier and allowing me to work on a great project in the esteemed organization.

I express my deepest thanks to my co-advisor Prof. Gael Thomas, Professor at Telecom SudParis, who along with Dr Marc Shapiro, in spite of being extraordinarily busy with his duties took the time to brainstorm, guide and keep me on the correct path for the successful implementation of the project objectives.

It's my radiant sentiments to convey my best regards and deepest gratitude to Dr Jonathan Lejeune, Associate Professor and Dr Julien Sopena, Associate Professor and my lab mates for their valuable inputs and research discussions.

I perceive the internship opportunity in the esteemed laboratory as an important milestone in my career development. I will strive to utilise the skills and knowledge gained for contributing to the research community to best of my abilities. I Hope to continue to collaborate with all of you in future.

Contents

1 Introduction **3**

2 Literature Review **4**
- 2.1 Memory Persistency (Pelley u. a., 2014) . . . 4
- 2.2 Mnemosyne (Volos u. a., 2011) . . . 4
- 2.3 NVML . . . 5
- 2.4 nvthreads (Hsu u. a.) . . . 5
- 2.5 Atlas (Chakrabarti u. a., 2014) . . . 5

3 Objective & Experimental implementations **6**
- 3.1 Objective . . . 6
- 3.2 Experiments with nvthreads . . . 6
  - 3.2.1 discussion . . . 6
  - 3.2.2 list structure . . . 7
  - 3.2.3 coarse grained list . . . 8
  - 3.2.4 fine grained list . . . 8
  - 3.2.5 open addressed *HASH-TABLE* implementation . . . 9
- 3.3 Experiments with Mnemosyne . . . 9
  - 3.3.1 discussion . . . 9
  - 3.3.2 *LIST* application . . . 10
  - 3.3.3 coarse grained list . . . 10
  - 3.3.4 fine grained list . . . 11
  - 3.3.5 closed addressed *HASH-TABLE* implementation . . . 11
- 3.4 Experiments with nvml . . . 12
  - 3.4.1 discussion . . . 12
  - 3.4.2 *LIST* application . . . 12
  - 3.4.3 coarse grained list . . . 13
  - 3.4.4 fine grained list . . . 14
  - 3.4.5 closed addressed *HASH-TABLE* implementation . . . 14

4 Evaluation **16**
- 4.1 *LIST* applications . . . 16
  - 4.1.1 evaluation setup . . . 16
  - 4.1.2 Mnemosyne, NVML and Volatile List . . . 16
  - 4.1.3 NVthread customised recovery evaluation . . . 19
- 4.2 hash table . . . 20
- 4.3 Reason for slow NVML . . . 20

5 Conclusion **21**

1 Introduction

Ever since I read about the computer memory system and the difference between volatile and non-volatile memories as a high school student and importance of saving work files before my computer goes out of power supply or crashes abruptly, I wondered why memory has to be volatile in the first place. Things got clear as I progressed further in my academics and I learned about memory hierarchy and its role in deciding the cost of the computer systems. I learned that to balance cost and performance tradeoff, the slower, cheaper and abundant non-volatile memory was kept furthest from CPU and fastest, expensive and smaller memory was kept closest to the CPU. This, in turn, has been a bottle neck as the data in order to become persistent and be available on next time the application is run, has to travel all the way down to memory hierarchy undergoing several marshalling and unmarshalling steps. This, of-course, was an overhead for the performance hungry high performance computing community. This problem emboldens when the storages systems happen to be in a distributed environment. Thus it is a serious performance bottle neck for in-memory computations. But recently proposed storage class memory(PCM, STT, Fe-RAM) which are supposed to have the performance like those of DRAM and persistence like those of Disks has given the community a hope of addressing the bottleneck, namely: 1). the performance overheads of the memory hierarchy and the 2). challenges of ensuring data persistence in case of system failures.

On the other hand, advent of NVRAM also has opened challenges like handling cache reordering, failure atomicity, ensuring easily adaptable programming abstractions, deciding when to make data durable and garbage collection. These challenges are exacerbated in presence of multiple threads, and programming model needs to decide upon the global consistent state of the application which can be made durable and at the same time ensures per thread consistency of logs or meta data, if any.

To summarise, this would allow leveraging performance of RAM like memory which is persistent too. To quote Jim Pappas, SNIA, Vice Chairman- "This is going to be the biggest change the computer architecture will go through, i.e. convergence of storage and memory", this open up the intelligent question of what should be the programming model to leverage this new persistent memory? such that its easier to program applications and at the same time ensuring consistency (Pelley u. a., 2014). To this end, many approaches have been proposed namely, Mnemosyne (Volos u. a., 2011), nvheaps (Coburn u. a., 2011), Atlas (Chakrabarti u. a., 2014), Intel's nvml and nvthreads(Hsu u. a.). During my stay in Regal Lab of INRIA, Paris as summer research intern I focused on building concurrent and persistent data-structure using the nvthreads, mnemosyene and nvml open source libraries. Precisely, I explored these libraries by building concurrent & persistent linked list. To implement such a list application I followed coarse grained and fine grained list implementation proposed by (Herlihy und Shavit, 2011). The set was made persistent by the programming abstraction provided by nvthreads, mnemosyne and nvml libraries. I also extended the list implementation to build concurrent and persistent hash-table. Finally, I evaluated the performance of the applications and explored the tradeoffs involved between ease of programming and efficiency. Major focus was to explore the efforts required on part of the programmer to program a concurrent data-structure for persistence and mechanisms adopted by the libraries, which in turn will help to better understand state of the

art.

2 Literature Review

2.1 Memory Persistency (Pelley u. a., 2014)

This paper builds upon the classical memory consistency models to propose analogous memory persistency models for NVRAM programming interfaces, to enable highly concurrent data structures which will be consistent in presence of failure. Authors argue that similar to memory consistency model where we have ordering constraints for loads and stores(over the volatile memory) to ensure consistency and permit more concurrency, along the same lines they define ordering constraints for persist(durable stores) operation to allow more concurrency, which would lead to minimizing the gap between the speed of stores over volatile memory and the speed of persist over persistent memory. To this end they propose 1) *persist buffering*: it allows durable stores to NVRAM to occur asynchronously allowing the thread to execute further computations. Thus, the latency of persist operations is allowed to overlap with a thread's useful execution, closing in much of the latency gap for durable stores and 2)*persist coalescing*: where two or more durable stores to a same memory block (a block can be of 8 byte granularity which current architectures support, to be modified atomically) are done within a single persist operation, for eg- if I have two persist operation to same memory block then equivalent single persist can be thought as if the only last one occured, ofcourse the ordering constraints have to be adhered. This reduces number of persists operations thus speeding up the application. The authors propose persistency model, namely *strict persistency*, *epoch persistency* and *strand persistency*, where unnecessary ordering between persist operations is relaxed successively.

2.2 Mnemosyne (Volos u. a., 2011)

Mnemosyne aims at providing programming abstraction to build persistent software applications over the SCM(or NVRAM). It provides this by extending linux virtual memory system to virtualize SCM- which enables multiple applications to run simultaneously in their own virtual memory segment which is persistent due to underlying SCM. It also modifies compiler/linker to provide transactional support for in-place update of large data structures(typically that of more than 8 Byte size). Thus, atomic blocks are instrumented into compilers to be extended as transactions using tinySTM. User can allocate static or dynamic memory using *pstatic*(compiler annotated) or *pmalloc* (persistent variant of malloc using memory mapping technique), respectively. CPU primitives like *flush* and *fence* are used to make data durable consistently avoiding problems of cache re-ordering. User can have persistent pointers (points to a desired memory chunk in SCM) using *persistent* keyword annotated as compiler attribute. However type safety mechanism is complex and depends on using not so readable macros. Using the programming abstraction of this library I implement a coarse grained and fine grained linked list.

2.3 NVML

NVML provides library support for programming persistent data structures without compiler/hardware modifications in a bunch of software libraries, I explored *libpmemobj* and its C++ binding. Libpmemobj lets a user create a persistent memory pool (which uses mmap) using unique name handles, This pool then can be converted to a usable pointer (which can be dereferenced directly) using inbuilt macros provided. To ensure failure atomicity of variable updates, the library provides transactions which are implemented using macros. The c++ binding hide much of the complexity and provides smart persistent pointers which can be used to implement a persistent data structure with classical c++ syntax.

2.4 nvthreads (Hsu u. a.)

Nvthreads provides persistent programming abstraction for multithreaded programs. It's threading mechanism is based on *Dthreads* and it depends on synchronization points(locks, unlocks) to ensure that data persists consistently on the SCM. It exposes the SCM as memory mapped regions and persistent variables can be allocated using nvmalloc(), with a unique name and size. The unique name provided during the memory allocation is used for accessing the allocated memory over subsequent run or crashes. The library maintains the location of the allocated variables in a persistent file(name varmap), where it stores name, offset and size of the variable allocated in persistent memory. It does not provide persistent pointers, so that users can directly access the persistent allocation without giving them a unique name handle. It also leaves it to the user to write a recovery routine and recover the persistent data-structures in case of crashes using the unique names provided during the allocation with help of exported nvrecover() function.

2.5 Atlas (Chakrabarti u. a., 2014)

ATLAS is another approach to provide programming support for NVRAM using synchronization primitives like lock/unlock. It is implemented with LLVM which instruments the synchronization operations to ensure failure atomicity during crash with help of persistent logs which guarantee consistency of the data structure stored in NVRAM. Logs track the critical sections and dependencies between them using the durability semantics of critical sections(FASE) laid down in the paper. It also exposes persistent memory regions using memory mapping and identifies the data structures stored in the persistent regions using the root object(name) which ensures that any object reachable from the root object remains durable and thus reachable across failures and multiple runs. Each thread maintains its own persistent log and consistency across multiple threads is ensured using the proposed durability semantics. *Visibility barriers* are used to make the data durable in NVRAM at each store update. Consistent or durable state is computed by a helper thread. Helper thread analyses the persistent log for calculating FASEs and a durability ordered graph is calculated, each node of this graph is a durable FASE with lock-unlock pair(this makes a durable node). And such a graph with all durable nodes is interpreted as a global consistent state.

3 Objective & Experimental implementations

3.1 Objective

Firstly, to implement a **concurrent** and **persistent** *LIST* application with the persistent memory programming libraries. The *LIST* application consists of *insert*, *del* and *find* operations. The *LIST* is implemented with linearizable linked list algorithms of (Herlihy und Shavit, 2011). I explored following two variants of the *LIST* implementation:

- *coarse-grained list*: The list has one global lock to be acquired by each operation sequentially.
- *fine-grained list*: The list has one lock for each node$< key, val >$ and allows more concurrent operations over the list by using hand-over-hand locking mechanism.

Secondly, to implement a **concurrent** and **persistent** *HASH-TABLE* application with the persistent memory programming libraries, namely Mnemosyne and NVML. The *HASH-TABLE* has to be built over the top of the previous *LIST* applications. Thus, the *HASH-TABLE* is closed addressed. And depending upon the type of the *LIST* application used *HASH-TABLE* can be of following types:

- *coarse-grained hash-table*: Each bucket is a coarse-grained list of $< key, val >$ pairs.
- *fine-grained hash-table*: Each bucket is a fine-grained list of $< key, val >$ pairs.

Thirdly, to evaluate the performance of the applications and finally to understand the challenges or trade-off involved in programming persistent and concurrent data-structures with NVML, Mnemosyne and NVML state of the art libraries from the perspective of efficiency and ease of programming.

3.2 Experiments with nvthreads

3.2.1 discussion

Nvthreads exports following functions to program a persistent application:

1. *nvmalloc:* This function takes size and an unique name of the persistent memory region to be allocated from a mmaped file.
2. *nvrecover:* This function aides to recover the memory allocated using the unique name provided during corresponding nvmalloc call.
3. *nvcheckpoint:* It is used to flush all the cache resident data to the NVRAM.

```
1    bool isCrashed(void);
2    unsigned long nvrecover(void *dest, size_t size, char *name);
3    void* nvmalloc(size_t size, char *name);
4    void nvcheckpoint(void);
```

It did not provide the persistent pointers, i.e. I cannot implicitly have a data structure containing a pointer field inside a nvmalloced datastructure such that I can recover the data on restart, for the simple reason that the address to which the

pointer points to goes invalid during next run as it is a virtual address. Though, there is a way to achieve this by explicitly maintaining the unique names of all the data structures I nvmalloc, and then manually reconstruct all the pointers during recovery using the unique names. For example, If I want to create a persistent linked list, I had to name each node uniquely and store this meta data either within the node or within a separate table in order to reconstruct the linked list during recovery. I modified the library recovery mechanism to implement a persistent linked list without naming each node uniquely, but the approach has a drawback that during recovery, the order of the recovered nodes depends on the order the nodes were allocated. Thus, reconstructing the linked list in the state it was before crash was tricky and inefficient (because library records the location of each nvmalloced node in a mmaped file and nodes can be recovered only in order they were written on the file). So, keeping in mind our aim of exploring the library for implementing persistent concurrent data structures we decided to implement a linked list where each node will be nvmalloced with a unique string name. Our experiments with concurrent implementation show that NVML could not ensure consistent durability.

3.2.2 list structure

We followed the fine grained and coarse grained concurrent list implementation provided in (Herlihy und Shavit, 2011). The list maintains two sentinel nodes *head* and *tail* with minumum and maximum *key* values. Each node's *key* lies within this range and all the nodes of the *LIST* are always sorted in increasing order of the *key*. The *LIST* allows: *insert*, *del* and *find* operations concurrently and ensures persistency using NVThread's programming semantics as discussed in section 3.2.1.

construction

The *LIST* is initialized using unique name(serves as root object to the persistent region) and its sentinel nodes are allocated memory with unique names. These names are stored in *headNodeName* & *tailNodeName* of the *LIST*, depicted as *list_t* in the structure below, this we call as persistent pointer information. Besides this, for efficiency purpose we have volatile *head* & *tail* pointers for regular list operations. This redundancy is worth having because using persistent names for normal list operation would be costly as it would involve repeated string reads and writes. So much for list meta data. Now, each regular *LIST* node maintains the non volatile pointer information using a structure which has *nextNodeName* and *selfNodeName*. These fields are used in reconstructing the list during recovery. Otherwise, regular *LIST* operations are performed using regular volatile next pointer, which goes invalid once program exits. The data structure maintained is as shown below:

```
typedef struct _next_nvpointer_info
{
  char nextNodeName[NODE_INFO_ARR_SIZE];
  char selfNodeName[NODE_INFO_ARR_SIZE];
} nvpointer_info_t;
```

```
typedef struct _node
{
  key_t key;
  val_t val;
  pthread_mutex_t node_mtx;

  nvpointer_info_t nvnext;
  struct _node *next;
} node_t;
```

```
typedef struct _list
{
  int node_count;
  int real_node_count;
  char headNodeName[NODE_INFO_ARR_SIZE];
  node_t *head;
  char tailNodeName[NODE_INFO_ARR_SIZE];
  node_t *tail;

  pthread_mutex_t gmtx;
} list_t;
```

persistence

Each thread which is a separate process infers durability semantics by intercepting lock-unlock pairs, and uses copy on write mechanism to buffer or commit modified memory pages to durable redo logs. It avoids ordering writes to NVM by just ensuring that writes to logs are ordered.

re-construction

isCrashed() function of the library tells whether the program has crashed earlier or not. After restart first thing the application needs to do is to reconstruct the *LIST*. To enable this, the library maintains the list of the location info(the unique name, page number, page offset and size) of all the nodes allocated, inside a file named *varmap* stored in the persistent region. Thus, upon recovery nvrecover() builds a map from the *varmap* file. I recover the root *LIST* structure by passing it's unique name to *nvrecover()* and then reconstruct the full list using the *nvpointer* structure by passing each node with the current node's *nextNodeName* member to the *nvrecover()*.

3.2.3 coarse grained list

The *LIST* has a global lock and each operation is synchronized using the *list-mutex*, depicted as gmtx in the list structure. The lock is regular pthread lock and do not persist across multiple run.

3.2.4 fine grained list

The *LIST* has a lock per node and each operation is synchronized using the *node-mutex*, depicted as node_mtx in the node structure. The lock is regular pthread lock and do not persist across multiple run. Fine grained list allows for more concurrency and only the region of the *LIST* where operation has to take effect is locked, i.e predecessor and successor node.

observations

In coarse-grained list, occasionally the recovery of some nodes fails as the *nextNodeName* fetched from recovered node is garbage. Seems in case of multiple threads logs written are not correctly made durable.

For fine-grained list, the persistence now depends on the nested critical sections as multiple locks are taken pair wise by multiple threads. And this puts the library to further stressful test. The library fails to recover the data after a crash even for a single threaded code. Though paper mentions that the nested critical sections are treated as single CS, but the list fails to persist correctly. Please find source code at (Singh, 2017a).

Moreover, amount of meta data and recovery code needed to be written dissuades programmer from using the library along with the fact that the application also is not consistent across failures. Please find source code here (Singh, 2017a)

3.2.5 open addressed *HASH-TABLE* implementation

The concurrent hash table implemented using the NVthreads is a simple linear probing based hash-table with *ht_add*, *ht_delete* & *ht_lookup* operations which are synchronized using global mutex lock. I allocate a single chunk of contiguous memory using *nvmalloc()*. Thus, the *hash-table* is simply an array of the $< key, value >$ pairs and contains no pointer members. The recovery for the hash table works correctly. The source code can be found at (Singh, 2017b).

For the meticulous readers, who might have observed by now that in objective I mentioned that I will be using the *LIST* to build the *HASH-TABLE*, my apologies for the disappointment. Since, the *LIST* failed to be consistent across multiple run, it would have been a bad idea to build a *HASH-TABLE* over such an inconsistent list using NVThreads. So, to compensate that I thought of building a simpler *HASH-TABLE* with NVThreads and ooh yes this time it worked :).

3.3 Experiments with Mnemosyne

3.3.1 discussion

Mnemosyne modifies kernel and have compiler instrumented code to support programming abstraction for the persistent data structure over NVM. It uses *scon* as a build tool instead of *make*. This involved a small learning curve for me, as I used it for the first time. We have used Mnemosyne's GCC version available at (Haris Volos und Swift, 2010). I had to install lots of dependencies to start using the Mnemosyne for building my applications. Mnemosyne provides following primitives to program a persistent application:

1. *pmalloc:* This function allocates the desired size of memory from the persistent heap on the SCM(here it is a mmaped file).

2. *persistent:* user can declare any type of variable as persistent and Mnemosyne understands that the variable is meant to be durable using compiler attributes.

3. *__transaction_atomic:* Allows in-place update of data-structures. As data more than 8 bytes donot get flushed atomically to the NVRAM/SCM it is required to ensure consistency of the data.

4. *pfree:* This function is meant to release the persistent region allocated.

3.3.2 *LIST* application

We followed the fine grained and coarse grained concurrent list implementation provided in (Herlihy und Shavit, 2011). The list maintains two sentinel nodes *head* and *tail* with minumum and maximum *key* values. Each node's *key* lies within this range and all the nodes of the *LIST* are always sorted in increasing order of the *key*. The *LIST* allows: *insert*, *del* and *find* operations concurrently and ensures persistency using Mnemosyne programming semantics as discussed in section 3.3.1.

construction

The *LIST* is declared as a global variable marked with MNEMOSYNE_PERSISTENT keyword and if it is not initialized already, then the *LIST* is initiated by allocating *head* and *tail* members a persistent region using *pmalloc*(). Multiple threads may invoke the *LIST* operations to populate it further, in coarse grained version the *list-mutex* is used to synchronize all the operations, where as the fine grained version has *node-mutex* (per thread mutex locks). The modification to the persistent nodes are made within lightweight durable transactions provided by Mnemosyne. These transaction provides failure safety, such that data becomes durable atomically.

```
typedef struct _node
{
        uint key;
        uint val;
        struct _node *next;
        pthread_mutex_t lmtx;
} mynode_t;
```

```
typedef struct _list
{
        uint node_count;
        mynode_t *head;
        mynode_t *tail;
        pthread_mutex_t gmtx;
} list_t;
```

persistence

Declaring the *LIST* (also a root to the persistent region) using the primitives of the library as discussed above ensures that data structure gets durable into SCM. The durable transactions ensure that the buffered logs are flushed consistently to the SCM/NVM.

re-construction

No explicit recovery code is required. The *LIST* which was declared statically(using MNEMOSYNE_PERSISTENT keyword which is instrumented into compiler with section attribute) as a root object can be recursively traversed via its next pointer field to access whole list across multiple runs of the application. One care though needs to be taken to reinitialize the mutex lock upon subsequent runs of the application. Otherwise, if a program crashes holding a lock, then same lock on the same node cannot be taken or program just may have indeterminate behaviour upon restart.

3.3.3 coarse grained list

The *LIST* has a global lock on the list and each operation is synchronized using the *list-mutex*, depicted as gmtx in the list structure. The lock is regular

pthread lock and do persist across multiple run along with other members of the structure. Source code can be found at (Singh, 2017c)

3.3.4 fine grained list

The *LIST* has a lock per node and each operation is synchronized using the *node-mutex*, depicted as lmtx in the node structure in section 3.3.2. The lock is regular pthread lock and persists its state across multiple run. Fine grained list allows for more concurrency and only the region(i.e predecessor and successor node) of the *LIST* where operation has to take effect is locked. Source code can be found at (Singh, 2017d)

observations

In coarse-grained list, the data structure remains consistent and persistent across multiple runs. However, while deleting a node using *pfree()* results in segmentation fault. On observing the dump problem seems to be at reinterpret_cast inside the library. The *pfree()* function should have worked as expected. This can be attributed to type safety handling of the library. The fine-grained list is consistently made durable across multiple run, but the problem with freeing the deleted node remains.

Mnemosyne provides persistent pointer support such that user can directly traverse through the pointer members of the persistent data structures and no explicit handling is required. However, it prohibits declaring a local variable as persistent and throws following exception : cannot set section attribute to local variables. As it instruments compiler using section attribute to decide whether allocation has to be provided from persistent region or the volatile. So, this is restriction programmers needs to be aware of while declaring its variables with persistent keyword.

3.3.5 closed addressed *HASH-TABLE* implementation

We build closed addressed hash table using both the variants of the list discussed above. Each bucket of the *HASH-TABLE* is a *LIST*. Coarse-grained hash-table is built using coarse-grained *LIST* and fine-grained hash-table is built using fine-grained *LIST*. we use *pmalloc()* to allocate persistent region equivalent to the *ht_size* of the type of the bucket(*list_t*). Following is the *HASH-TABLE* structure.

```
typedef struct _root
{
  uint ht_size;
  pthread_mutex_t htmtx;
  list_t *ht ;
} root_t;
```

Both the fine-grained and coarse-grained variants of the persistent and concurrent *HASH-TABLE* worked correctly.

3.4 Experiments with nvml

3.4.1 discussion

NVML provides c++ bindings to program NVM/SCM. It involves no changes to compiler or kernel and do not require any extra hardware support. It simply makes excessive use of power of MACROS to implement type safety and transactional system for in-place updates to the underlying NVM device. The tutorials and online support available for the library along with its designs of API makes it more convenient to use and program with, especially for the explorers who just want to explore programming with NVM. I used *libpmemobj* library to write my concurrent and persistent applications as described in section 3.1.

1. *make_persistent:* Allows declaring a typed variable meant to be persistent in a memory mapped file.

2. *persistent_ptr:* Allows declaring a persistent pointer.

3. *transaction::exec_tx:* Allows in place update of data structures into the SCM/NVM. It uses lambda expression a feature of C++.

4. *delete_persistent:* Allows to delete an allocation from persistent region(essentially a mmaped file provided by programmer).

3.4.2 *LIST* application

We followed the fine grained and coarse grained concurrent list implementation provided in (Herlihy und Shavit, 2011). The list maintains two sentinel nodes *head* and *tail* with minumum and maximum *key* values. Each node's *key* lies within this range and all the nodes of the *LIST* are always sorted in increasing order of the *key*. The *LIST* allows: *insert*, *remove* and *find* operations concurrently and ensures persistency using NVML programming semantics as discussed in section 3.4.1. It provides persistent pointer support such that user can directly traverse through the pointer members of the persistent data structures and no explicit handling is required. The programming interface provides interactive type safety mechanism.

construction

The programmer needs to create a persistent object pool which is simply an abstraction for providing a memory mapped region. Using the pool::create() user provides the unique name, file, size and type of the root object to get allocated a persistent pool. From this pool the application can allocate and deallocate its persistent regions throughout its lifetime using the root object returned by pool::create(). So, the application checks before starting that weather it has a pool already existing and if a pool is already allocated then user can get a pointer to the root of the pool using pool::open(). And then using pool::get_root() user can fetch a directly usable pointer for the pool. The list can then be populated by its operations using the primitives provided by the library, discussed in section 3.4.1. Each variable which is meant to be persistent is declared with *p* alongwith its type as in c++ style. And every pointer to persistent memory can be declared through smart pointer support as shown in the code below. The locks global_list_pmutex(for coarse grained variant) and local_list_pmutex(for

fine grained variant) are automatically re-initialized across multiple runs, unlike Mnemosyne. Thus, we do not need to worry about the lock states during recovery.

```
class pmem_list
{
/* entry in the list */
  struct pmem_entry {
    p<uint_t> key;
    p<uint_t> val;
    mutex local_node_pmutex;
    persistent_ptr<pmem_entry> next;
   };

 persistent_ptr<pmem_entry> head;
 persistent_ptr<pmem_entry> tail;
 mutex global_list_pmutex;

public:
 void init (pool_base &pop);
 bool insert(pool_base &pop, uint_t key, uint_t val);
 bool remove(pool_base &pop, uint_t key, uint_t* val);
 bool find(pool_base &pop, uint_t key, uint_t* val);
};
```

persistence

The *LIST* is declared as a root with unique name(handle), which mmaps a file to provide a persistent residence for the data accessible via root. All persistent data is modified within the durable transactions, which maintains a per thread undo log and underlying architectures CLFLUSH, CLWB ensure that data reaches persistent storage upon commit. The transactions are not thread safe, reason being the complexity involved in maintaining consistent view of updates by transactions in case of multi-threading. Each transaction uses significant amount of memory for logs and metadata such that even 8MB of memory is not enough for a transaction with 3 nodes in the *LIST*.

re-construction

No explicit recovery code is required. The *LIST* which was declared as a root object can be recursively traversed via its next pointer field to access whole list across multiple runs of the application. The program can simply start running with a *LIST* state as it was when crash occurred. The list state would be the one of last successful transaction which changed the state of the *LIST*.

3.4.3 coarse grained list

The *LIST* has a global lock on the list and each operation is synchronized using the *list-mutex*, depicted as global_list_pmutex in the list structure. The lock is C++ mutex lock and NVML ensures the initilization of locks across multiple runs and crashes. Source code can be found at (Singh, 2017e)

3.4.4 fine grained list

The *LIST* has a lock per node and each operation is synchronized using the *node-mutex*, depicted as local_list_pmutex in the node structure. Fine grained list allows for more concurrency and only the region of the *LIST* where operation has to take effect is locked, i.e predecessor and successor node. Please refer to source code here (Singh, 2017f)

observations

Coarse-grained *LIST* works correctly across multiple runs, and is easy to program. However, it's performance is slow owing to the reason that it excessively depends upon the MACROS and shared libraries and also excess number of IO operations at kernel level, we discuss this in detail in section 4 . The flush and mfence instructions used for ensuring durability also contribute to its slow performance as unlike Mnemosyne these logs and flushing is not optimized. This is fairly understandable as purpose of library is to provide easy to use persistence programming to the user straightaway with the existing architectural support.

The fine-grained *LIST* is also consistently made durable across multiple runs. One of the most appreciable aspect of the NVML is that programmer need not worry about manual initiation of the pthread locks taken for synchronization. If the locks are member of a persistent data structure, like the locks in our list and node structure which are allocated from persistent regions, they are implicitly inited and user does not have to worry about them. Also, if application happens to crash while holding locks they are implicitly unlocked which is not the case with Mnemosyne.

3.4.5 closed addressed *HASH-TABLE* implementation

We build closed addressed hash table using both the variants of the list discussed above. Each bucket of the *HASH-TABLE* is a *LIST*. Coarse-grained hash-table is built using coarse-grained *LIST* and fine-grained hash-table is built using fine-grained *LIST*. we use *make_persistent* to allocate persistent region equivalent to the *ht_size* of the type of the bucket(*pmem_list*). Following is the *HASH-TABLE* structure.

```
class pmem_ht {

        struct pmem_entry {
                p<uint_t> key;
                p<uint_t> val;
                mutex local_node_pmutex;
                persistent_ptr<pmem_entry> next;
        };

        struct pmem_list{
                persistent_ptr<pmem_entry> head;
                persistent_ptr<pmem_entry> tail;
                mutex global_list_pmutex;
        };

        p<bool> is_init = false;
```

```
        p<bool> ht_size = tab_size;
        persistent_ptr<pmem_list[]> ht;

public:
        uint_t hash_func(uint_t key);
        void init (pool_base &pop);
        bool insert(pool_base &pop, uint_t key, uint_t val);
        bool remove(pool_base &pop, uint_t key, uint_t* val);
        bool find(pool_base &pop, uint_t key, uint_t* val);
        bool is_inited(void);
};
```

Both the fine-grained and coarse-grained variants of the persistent and concurrent *HASH-TABLE* worked correctly.

4 Evaluation

4.1 *LIST* applications

4.1.1 evaluation setup

The performance of Mnemosyne, NVML and simple volatile list is evaluated for coarse grained as well as fine grained version. We measure wall clock time of the average time taken by all the operations (*insert*, *del* and *find*). Average is taken over 50 runs for each of 10, 20, 30, 40 and 50 number of threads executing the application. We plot average time taken vs total number of threads graph. Each thread is designated to perform a single operation and load distribution is as follows: 50% of the threads are assigned *insert* operation, 40% of the threads are assigned *del* operation and 10% threads are assigned *find* operation. The application's root were re-inited after every 2 runs. Thus, in first run the application intializes the root and carries out all the operation and in second run the application carries out newer set of operation after recovering the previous list of the first run.

- glibc version: Ubuntu GLIBC 2.23-0ubuntu9
- Architecture: x86_64, Intel(R) Core(TM) i3-6100U CPU @ 2.30GHz.
- number of CPU: 4

4.1.2 Mnemosyne, NVML and Volatile List

coarse grained

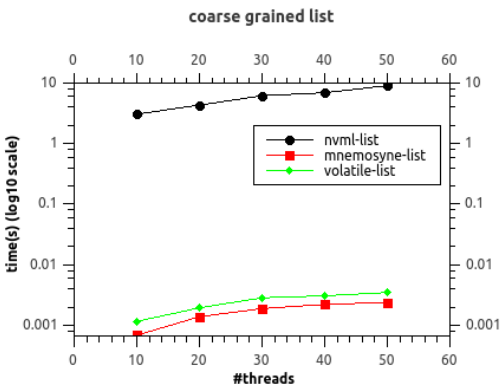


Figure 1: average time vs number of threads on log_{10} scale.

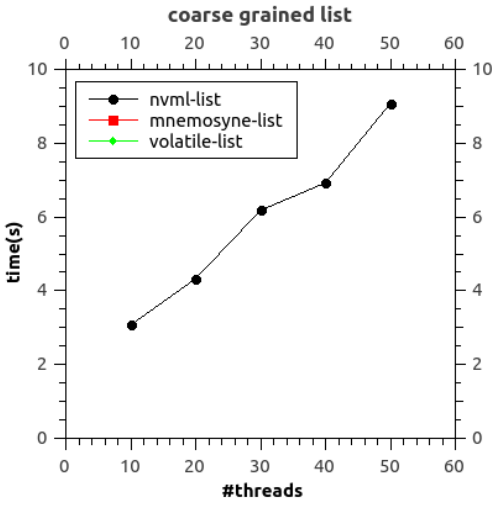


Figure 2: average time vs number of threads on linear scale.

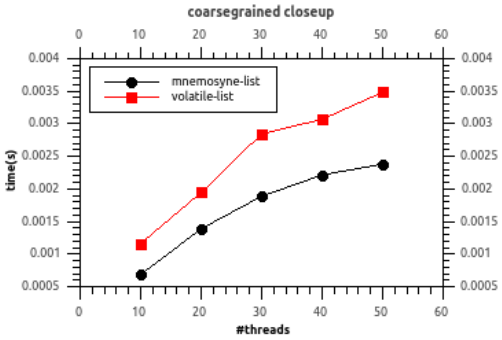


Figure 3: average time vs number of threads, close view of Mnemosyne and volatile list.

fine grained

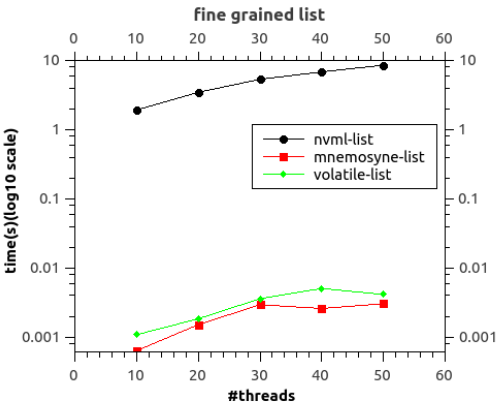


Figure 4: average time vs number of threads on log_{10} scale.

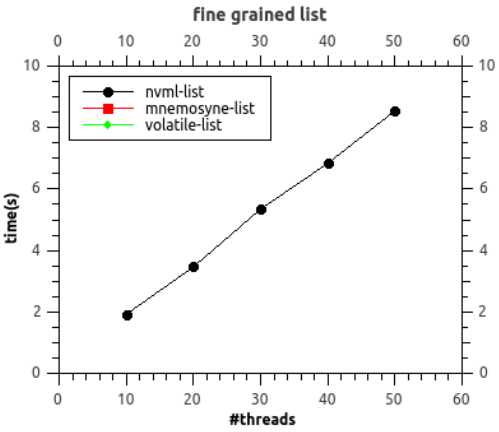


Figure 5: average time vs number of threads on linear scale.

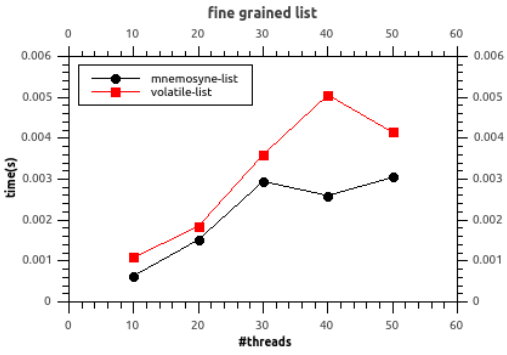


Figure 6: average time vs number of threads, close view of Mnemosyne and volatile list.

Individual library specific comparison

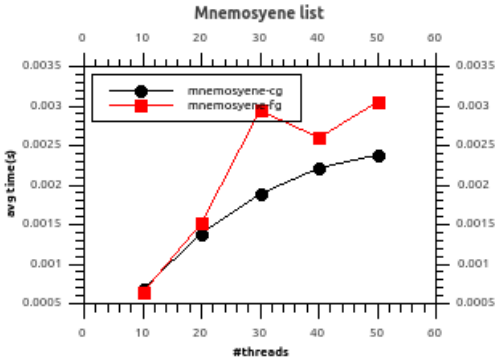


Figure 7: Mnemosyne fine grained and coarse grained evaluation.

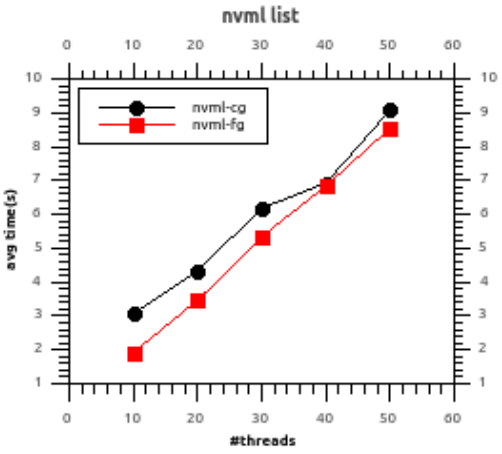


Figure 8: NVML fine grained and coarse grained evaluation.

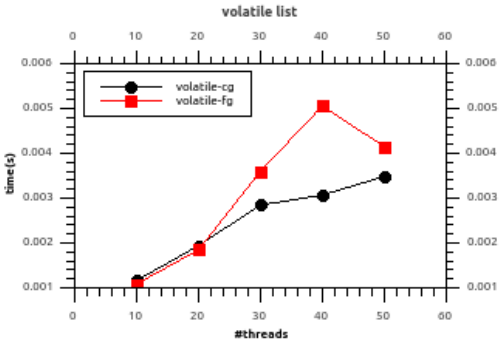


Figure 9: volatile fine grained and coarse grained evaluation.

Evaluation for cumulative run of the applications

In this evaluation we allow application to run multiple times each time recovering the previous *LIST* and applying new set of operation over this *LIST*. number of threads are 50 Rest of the evaluation setup remains same as in section 4.1.1.

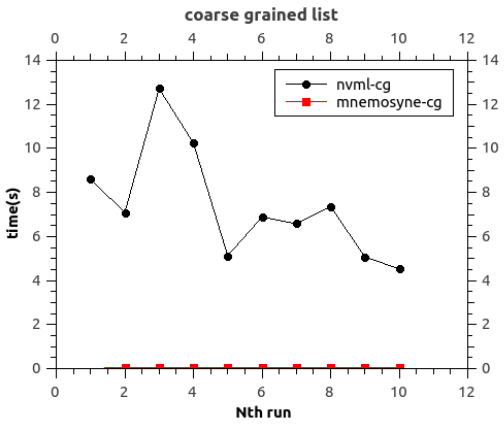


Figure 10: Coarse grained *LIST* evaluation for multiple runs.

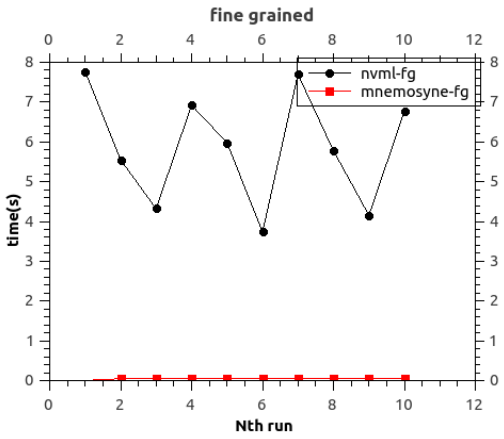


Figure 11: Fine grained *LIST* evaluation for multiple runs.

4.1.3 NVthread customised recovery evaluation

I customized the recovery code of the NVML and used a multimap to log the location of the persistent nodes representing a *LIST* with same name and upon recovery sorted the nodes to maintain the invariant of sorted nodes. complete code can be found here at (Singh, 2017g). Implementation of hash table was straight forward as it does not require any persitent pointer field. Hash table is open addressed linear probing hash table. We didnot explore much for evaluating Nvthreads as our focus was to implement concurrent and persistent data-structure but with NVthreads the consistency couldn't be achieved as discussed in section 3.2. SO with much of tricks and efforts we just could implement basic data structures.

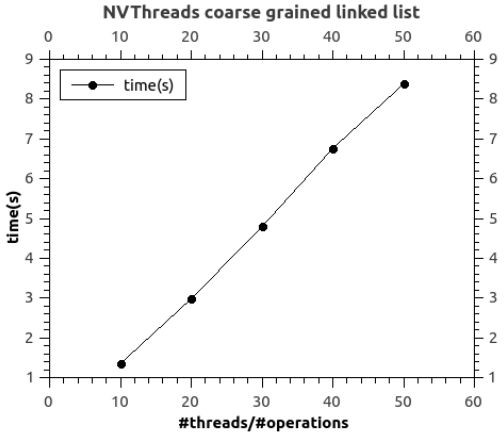


Figure 12: Average time for coarse grained linked list

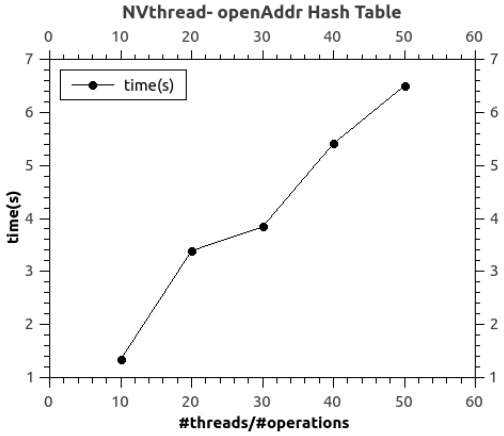


Figure 13: Average time for coarse grained hash table.

4.2 hash table

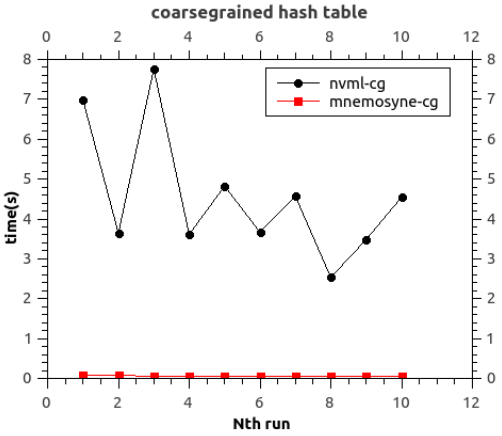


Figure 14: Time for coarse grained linked list over 10 runs with 50 threads

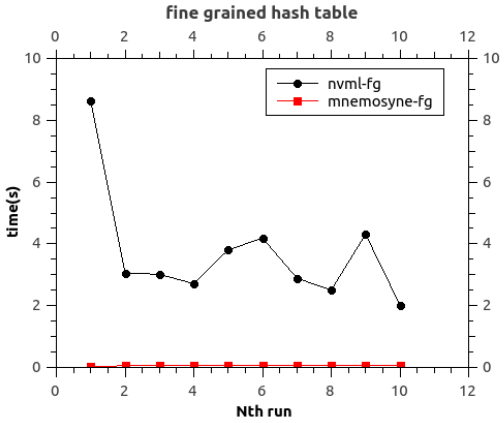


Figure 15: Time for fine grained hash table over 10 runs with 50 threads.

4.3 Reason for slow NVML

We have observed that Mnemosyne performs way faster than the NVML counterparts. This can be attributed to efficient logging and paging mechanism of the Mnemosyne implemented via compiler instrumentation. But NVML has just focused on ease of programming aspect without caring much about the efficiency and optimization, to keep the library simple. Moreover, we dig deeper, it is observed that NVML spends lot of time in doing IO operation in comparison to Mnemosyne. We validate this through *iostat* utility of linux to plot number of KB written per second and number of transfers requested per second to the underlying device for the applications running with NVML and Mnemosyne. We measure this for coarse-grained *LIST* only as this gives enough insight for the behaviour of other application variants as well.

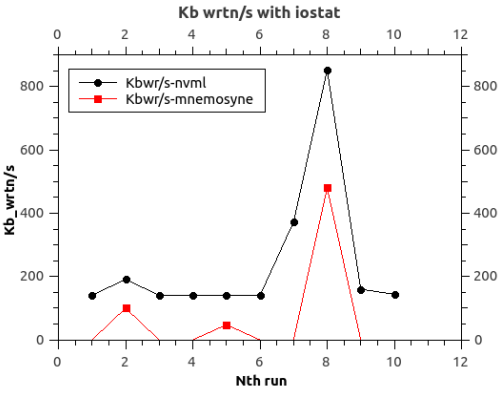


Figure 16: Number of KBs of data written/sec to the underlying NVM device(memory mapped file) over 10 runs with 50 threads

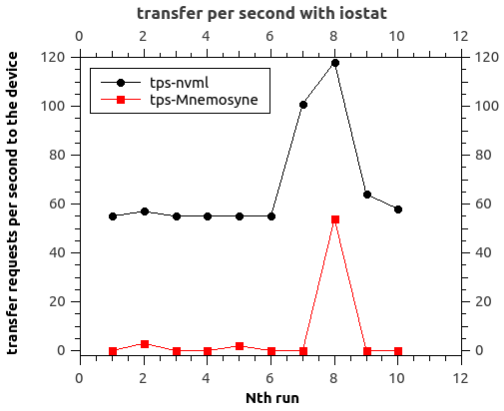


Figure 17: Number of transaction per second requested to memory mapped file(in memory FS) over 10 runs with 50 threads.

5 Conclusion

I wrote persistent fine-grained and coarse-grained list using the programming models of NVML, NVthread and Mnemosyne and existing volatile coding mechanism. Over the top of these two *LIST* variants, for each of the programming libraries, I also implemented the *HASH-TABLE* application. Finally, I studied their performance and evaluated them in section 4.

Mnemosyne applications worked correctly and its performance (as can be seen from the plots) is the best amongst all the libraries subjected to the study. However, it can be attributed much to the compiler specific changes and kernel modifications. In the case of Mnemosyne, type safety of the persistent data structures is complex and may be error prone. Also freeing the deleted node's memory failed for the applications due to type casting run time error. Thus, garbage collection did not work for the application. I had to write the recovery code section specifically to re-initialize the locks taken before the crash of the application.

NVThread's approach was most tricky to program with as it did not support the persistent pointers, so both fine grained and coarse grained lists programmed through it failed to recover some of the nodes because the recovery meta-data got corrupted. However, the open addressed hash table worked correctly as it only involved a contiguous memory allocation without the use of any pointer member variable and simple coarse grained locks for synchronization. Also, it is not clear how a fine grained list like the one with hand over hand locking could be built efficiently, as the NVThreads approach treats nested locks equivalent to one single critical section. It also required the significant amount of recovery code. Also, The garbage collection did not work for the deleted nodes.

On the other hand NVML though being very slow in terms of performance was easiest to program with and required no recovery code for the two concurrent list variants. Mnemosyne has been the quickest of all for obvious reasons - given it implements own virtual paging mechanism and compiler modification also it implements an efficient logging mechanism for its durable transaction to update data atomically, which reduces performance cost significantly. NVML (as a design decision) has completely focused on ease of programming the application and implements all the feature at the software level in form of libraries, with the heavy dependency on macros. NVML's durable transactions, in fact, uses MBs of meta data and uses hardware persistence primitives multiple times to flush data to NVM device.

References

[Chakrabarti u. a. 2014] Chakrabarti, Dhruva R. ; Boehm, Hans-J ; Bhandari, Kumud: Atlas: Leveraging locks for non-volatile memory consistency. In: *ACM SIGPLAN Notices* 49 (2014), Nr. 10, S. 433–452

[Coburn u. a. 2011] Coburn, Joel ; Caulfield, Adrian M. ; Akel, Ameen ; Grupp, Laura M. ; Gupta, Rajesh K. ; Jhala, Ranjit ; Swanson, Steven: NV-Heaps: making persistent objects fast and safe with next-generation, non-volatile memories. In: *ACM Sigplan Notices* 46 (2011), Nr. 3, S. 105–118

[Haris Volos und Swift 2010] Haris Volos, Andres Jaan T. ; Swift, Michael M.: *MNEMOSYNE*. https://github.com/snalli/mnemosyne-gcc. 2010

[Herlihy und Shavit 2011] Herlihy, Maurice ; Shavit, Nir: *The art of multiprocessor programming*. Morgan Kaufmann, 2011

[Hsu u. a.] Hsu, Terry Ching-Hsiang ; Brügner, Helge ; Roy, Indrajit ; Keeton, Kimberly ; Eugster, Patrick: Practical Persistence for Multi-threaded Applications.

[Pelley u. a. 2014] Pelley, Steven ; Chen, Peter M. ; Wenisch, Thomas F.: Memory persistency. In: *Computer Architecture (ISCA), 2014 ACM/IEEE 41st International Symposium on* IEEE (Veranst.), 2014, S. 265–276

[Singh 2017a] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-nvthreads-experiments/uniqueNameapproach/ArtOfMultiproccssorsAlgos/myapps. 2017

[Singh 2017b] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-nvthreads-experiments/custom-recovery-approach/myapps/open-addressed-hashtable. 2017

[Singh 2017c] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-mnemosyne-experiments/usermode/examples/coarse-grained-plist. 2017

[Singh 2017d] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-mnemosyne-experiments/usermode/examples/fine-grained-plist. 2017

[Singh 2017e] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-nvml-experiments/src/examples/libpmemobj/cpp/coarsegrained-llist. 2017

[Singh 2017f] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-nvml-experiments/src/examples/libpmemobj/cpp/finegrained-llist. 2017

[Singh 2017g] Singh, Ajay: *persistent data structure experiments*. https://github.com/aajayssingh/concurrent-persistent-datastructure/tree/master/my-nvthreads-experiments/custom-recovery-approach/myapps. 2017

[Volos u. a. 2011] Volos, Haris ; Tack, Andres J. ; Swift, Michael M.: Mnemosyne: Lightweight persistent memory. In: *ACM SIGARCH Computer Architecture News* Bd. 39 ACM (Veranst.), 2011, S. 91–104